A Beginner's Guide to Retrieval-Augmented Generation

An original introduction written for this sample corpus.

Free to use for learning and testing.

1. Introduction

Large language models can write fluent answers, but they only know what was in their training data. They have never read your lecture notes, your saved articles, or the book you finished last month. Retrieval-augmented generation, usually shortened to RAG, closes that gap. Instead of asking the model to remember everything, the system looks up the most relevant passages from your own documents and hands them to the model together with the question.

A RAG system therefore has two halves. The first half prepares the documents once: it reads the files, cuts them into small pieces, and stores them in a form that can be searched quickly. The second half runs for every question: it finds the best pieces, builds a prompt around them, and asks the model to answer using only that material. Because the pieces keep their source and page number, the answer can include citations that the reader can check.

This guide walks through each stage in plain language. It is written for students who are comfortable with programming but new to applied machine learning, and it focuses on the decisions that matter most in practice rather than on mathematical detail.

2. How Embeddings Work

An embedding is a list of numbers that represents the meaning of a piece of text. A small embedding model reads a sentence or a paragraph and produces, for example, 384 numbers. Texts with similar meanings produce similar lists, so the words car and automobile end up close together even though they share no letters. This is what allows a system to search by meaning instead of by exact words.

To compare two embeddings we measure the angle between them, a measure called cosine similarity. A value near one means the texts are very close in meaning, and a value near zero means they are unrelated. A vector database stores millions of these lists and can quickly return the ones closest to a query. Indexes such as HNSW make this fast by organising the vectors into a graph that can be searched without comparing against every item.

Embeddings are not perfect. They can miss exact identifiers such as error codes, names, and dates, because those carry little general meaning. A query for a specific version number may return passages that are merely about versions in general. For this reason, many systems combine embedding search with ordinary keyword search, a combination known as hybrid search.

The choice of embedding model is a trade-off between quality, speed, and size. Small models run comfortably on a laptop processor and are good enough for a personal collection of a few hundred documents. Larger models give better results on difficult queries but need more memory and time. Whatever model is chosen, the same model must be used for the documents and for the questions, because vectors from different models cannot be compared.

3. Chunking Documents

Documents are too long to embed or to place into a prompt as a whole, so they are cut into chunks. A chunk should be large enough to carry a complete idea and small enough that a search result stays focused. Chunks of roughly two hundred to three hundred words are a common starting point, because they are about the size of a few paragraphs.

The simplest method cuts the text every fixed number of words, but this often splits a sentence or an explanation in the middle. A better method respects the structure of the document. It splits on headings first, then on paragraphs, and only on sentences when a paragraph is still too long. Following the structure keeps related sentences together and makes each chunk easier to understand on its own.

Overlap between neighbouring chunks is a useful safety net. If each chunk repeats the last few sentences of the previous one, an idea that falls across a boundary is still fully present in at least one chunk. Too much overlap wastes space and produces near-duplicate results, so a small overlap of around forty words is usually enough.

Every chunk should also carry metadata: the file it came from, the page number, the nearest heading, and the date of the document. This information is not used for the embedding itself, but it is what makes citations possible, and it allows questions such as the book I read last month to be answered by filtering on dates.

4. Retrieval and Reranking

When a question arrives, the system first retrieves a generous set of candidates, perhaps thirty chunks, using both vector search and keyword search. The two result lists are then merged. A simple and effective merging method is reciprocal rank fusion, which gives each chunk a score based on its position in each list and adds the scores together. Chunks that rank well in both lists rise to the top.

The candidates are then passed to a reranker. Unlike the embedding model, which reads the question and each chunk separately, a reranker reads them together and produces a more careful relevance score. This is slower, which is why it is applied only to the shortlist. In practice a reranker often gives the largest single improvement in answer quality for the least code.

Finally the best few chunks, typically five, are placed in the prompt with numbered labels. The model is told to answer only from those sources and to cite them by number. If the retrieved material does not contain the answer, the system should say so plainly rather than guess. A score threshold on the reranker output is a simple way to detect this situation.

Questions that span several documents need extra care. A comparison between two books, for instance, can be split into one sub-question per book, with retrieval run separately for each, and the results combined before the model writes the final answer. This technique is called query decomposition and is one way a system can go beyond simple search.

5. Evaluating the System

It is easy to build a demo that looks impressive on three hand-picked questions and fails on everything else. The remedy is an evaluation set: a list of thirty to forty real questions, each paired with the passage that should be found. After every change to the system the evaluation is run again, and the scores show whether the change helped.

Two metrics are widely used. Hit rate at five measures how often the correct passage appears among the top five results. Mean reciprocal rank measures how high the correct passage is placed, giving more credit for rank one than for rank five. Reporting both before and after each improvement makes the effect of a design choice visible and gives concrete numbers to discuss.

The evaluation set should include a few questions that the documents cannot answer. A trustworthy assistant must be able to respond that it found nothing relevant. When scores are low, check retrieval first: if the right passage is not retrieved, no prompt can rescue the answer, whereas if it is retrieved and the answer is still wrong, the problem lies in generation.

6. Privacy and Security

Personal documents are sensitive, so the design should keep data on the owner's machine whenever possible. Running the embedding model and the language model locally means that no text leaves the computer. If a hosted model is used for better answers, only the few retrieved passages should be sent, and this should be stated clearly to the user.

Documents are untrusted input. An email or a saved web page can contain text that tries to give the model instructions, an attack known as prompt injection. The prompt should state that the sources are data to be quoted, never orders to be followed, and the application should never take actions based on what a document says.

Users need control over their data. The system should let them exclude folders from ingestion and delete a document completely, including its chunks and vectors. Logs should record identifiers rather than document text, so that a log file never becomes a second copy of private information. Secrets such as database passwords belong in environment files that are excluded from version control.